System Architecture: PermitParser Ingestion Pipeline

Date: September 24, 2025

1. Executive Summary

1.1. System Purpose

The primary requirement is to build a system that automatically helps customers discover and bid on construction **Projects**.

To do so, it scrapes public documents from municipal sources and transforms their unstructured content into a granular, indexed, and queryable format within Google Cloud Platform (GCP). The entire system is designed around the **Project** as the central business entity.

1.2. Core Architectural Challenge & Solution

The system's workflow presents conflicting technical requirements. The initial analysis of a document is an inherently **sequential** process that must be transactional.

However, the computationally expensive parsing of document content must be performed in a **massively parallel** fashion to be efficient. Finally, the aggregation of these parallel results into a single index is a common point of failure, risking **data corruption** through race conditions.

To solve this, the proposed architecture is a **Hybrid Orchestration-Choreography Model**. It deliberately uses the right tool for each stage of the job:

- **Google Cloud Tasks (Orchestration):** Manages the transactional, sequential steps (initial decomposition and final aggregation) to guarantee reliability and prevent race conditions.
- **Google Cloud Pub/Sub (Choreography):** Manages the stateless, parallel parsing stage to achieve maximum performance and scalability.

This hybrid design provides a production-ready solution that delivers high performance, cost efficiency, and the transactional integrity required for a reliable data ingestion pipeline.

2. High-Level Architecture & Technology

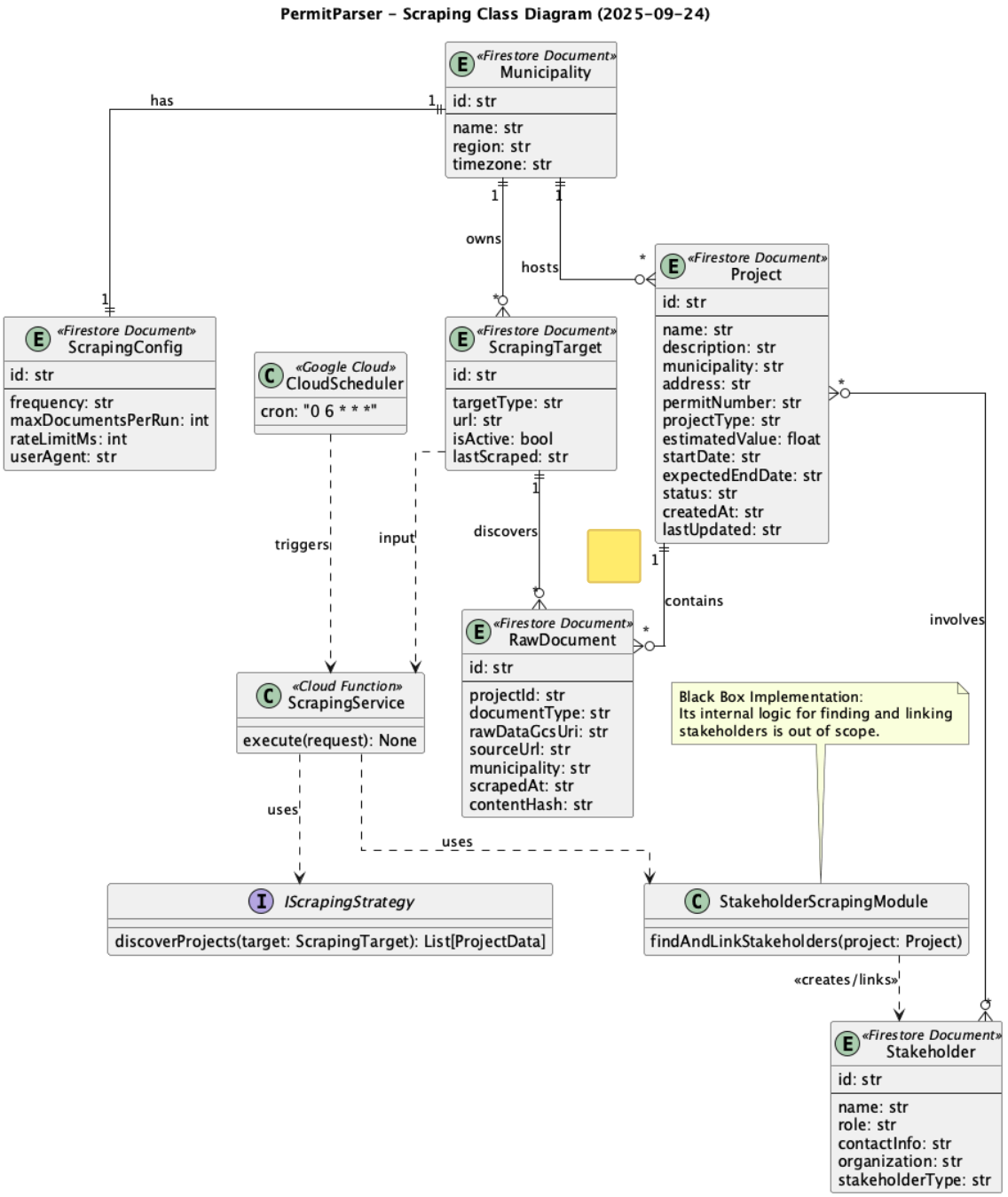
2.1. Technology Stack

- **Programming Language**: Python 3.11+
- **Cloud Platform**: Google Cloud Platform (GCP)
- **Containerization**: Docker
- **Architecture Pattern**: Hybrid Orchestration-Choreography
- **Metadata/Index Database**: Firestore for **Projects**, **Stakeholders**, and processing indices.
- **Data Object Storage**: Google Cloud Storage (GCS) for raw documents and large parsed data objects.
- **Messaging**: Pub/Sub for parallel processing + Cloud Tasks for reliable transactions.
- **Compute**: Cloud Run & Cloud Functions for auto-scaling services.

3. Detailed Architectural Breakdown

The system is composed of two primary domains—Scraping and Processing—connected by a reliable, transactional handoff that maintains project context throughout.

3.1. Scraping Domain: Construction Project Discovery



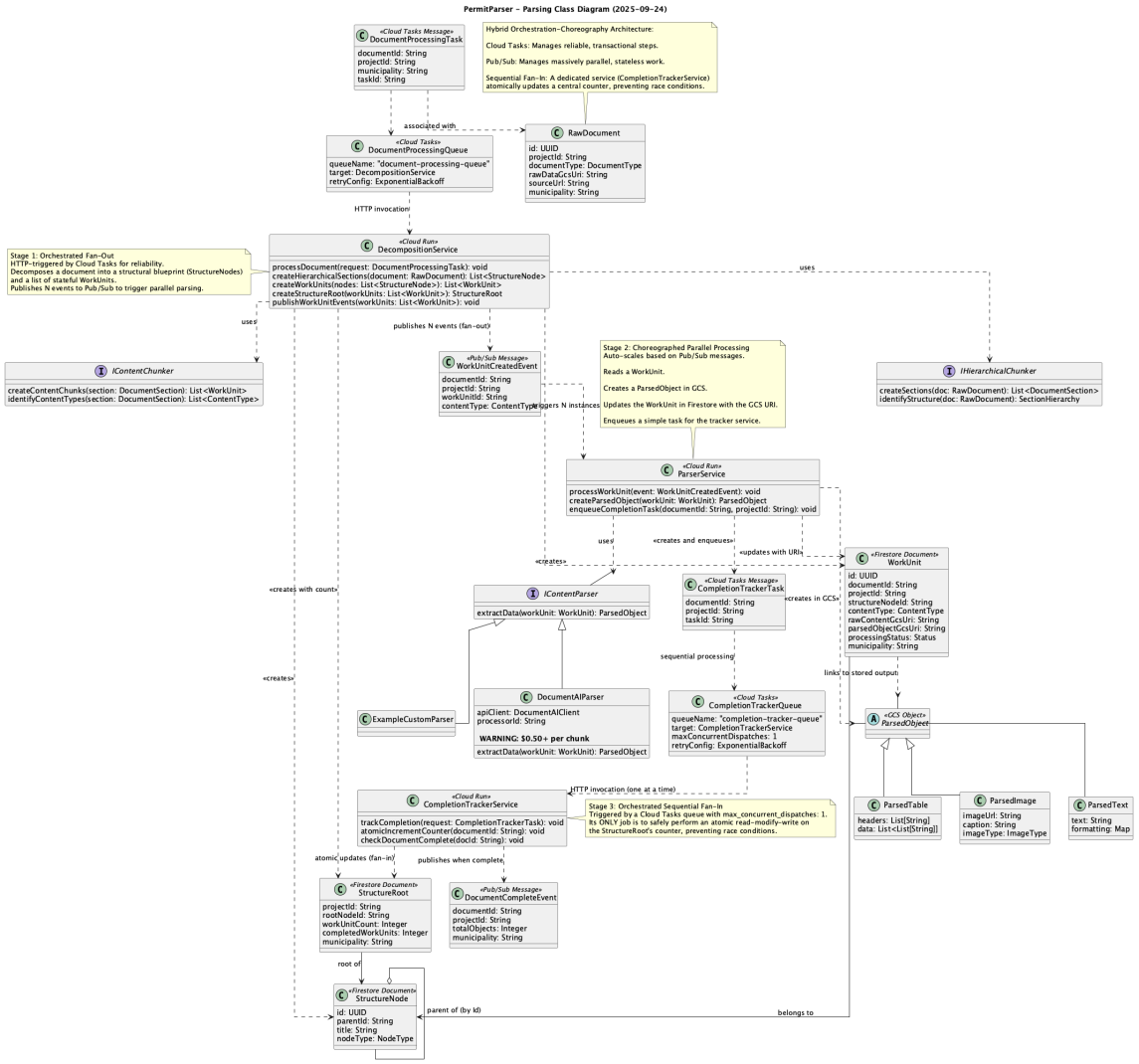
This domain is responsible for discovering and ingesting **construction projects** with their associated documents and stakeholders.

Project-Centric Architecture: The system scrapes construction projects, not just municipal documents. Each scraped entity represents a real-world construction project with permits, stakeholders, and multiple documents.

- **Initiation:** A **Cloud Scheduler** job triggers the process on a defined schedule.
- **Project Discovery:** An HTTP-triggered **ScrapingService** (Cloud Function) receives the request and uses a **Strategy Pattern** to select the appropriate scraper for the target municipality.

- **Project Creation:** The scraper identifies construction projects from municipal sources and creates a `Project` entity in Firestore with metadata (permit number, address, estimated value, project type, etc.).
- **Document Association:** For each project, the scraper fetches **all associated documents** (permits, plans, approvals) and stores them in GCS. It then creates a `RawDocument` record in Firestore for each, ~~linking it to its parent `Project` via the `projectId`~~.
- **Stakeholder Handling:** The system includes a **Stakeholder Scraping Module** (implementation TBD) that can identify and associate stakeholders (contractors, architects, property owners) with projects. This module is designed as a black box to avoid scope creep, but the data model supports it.
- **Handoff:** The `ScrapingService` creates a `DocumentProcessingTask` for each discovered document in a **Cloud Tasks** queue. Crucially, each task includes the `projectId`, ensuring the processing pipeline is reliably triggered with the proper business context.

3.2. Hybrid Parsing Pipeline



This is the core of the system, designed in three stages for performance and data integrity. **Project context is maintained at every step.**

Stage 1: Orchestrated Decomposition (Transactional Fan-Out)

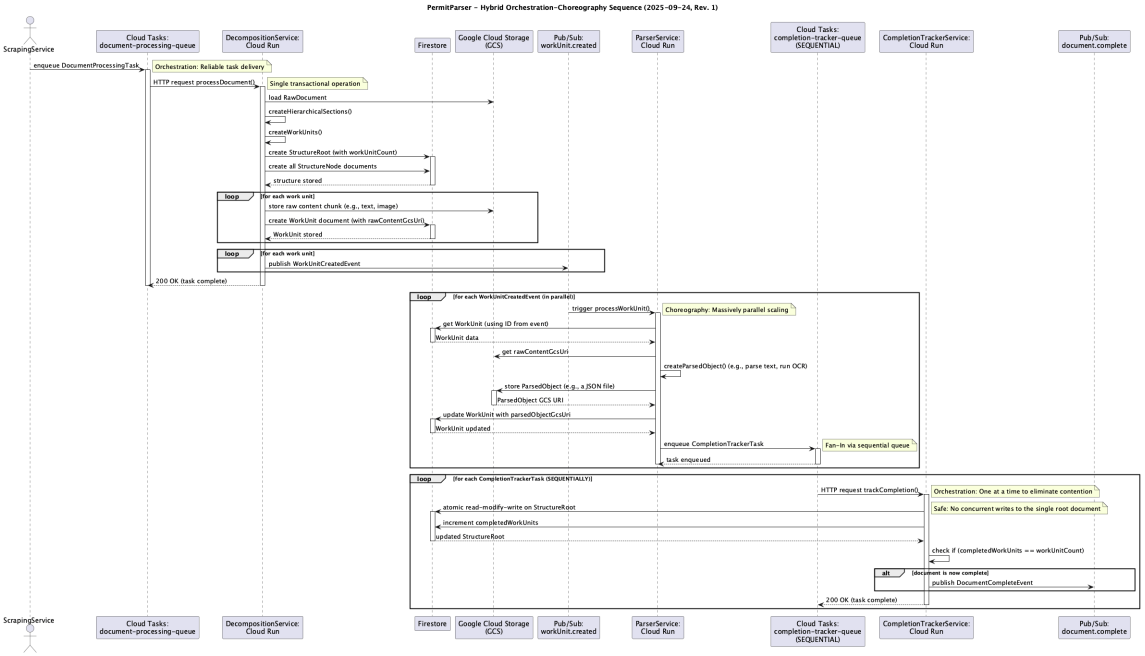
- **Rationale:** Document structure analysis is a sequential, atomic operation. This stage establishes the document's hierarchical index and the total work unit count needed for progress tracking, while maintaining project context.
- **Flow:**
  1. Cloud Tasks delivers the `DocumentProcessingTask` (containing `documentId` and `projectId`) to the **DecompositionService** (Cloud Run).
  2. The service reads the original document from GCS. For each logical section, it extracts the raw content and saves it as a new, separate physical file (e.g., a `.txt` or `.png`) in a "raw work units" GCS bucket.
  3. It then creates all `StructureNode` (the blueprint) and `WorkUnit` (the work ticket) documents in Firestore. Each `WorkUnit` is stamped with the `projectId`.
  4. It creates the primary `StructureRoot` metadata document, crucially populating the `workUnitCount` and stamping it with the `projectId`.
  5. **Fan-Out:** The service publishes N individual `WorkUnitCreatedEvent` messages to a Pub/Sub topic, one for each work unit, each carrying the `projectId` and `documentId`.

Stage 2: Choreographed Parallel Parsing

- **Rationale:** Parsing individual work units is computationally expensive, stateless, and perfectly suited for massive parallelization. Project context is maintained throughout processing.
- **Flow:**
  1. Pub/Sub delivers `WorkUnitCreatedEvent` messages (with `projectId` context) to a dynamically scaling pool of **ParserService** instances (Cloud Run).
  2. Each instance reads a `WorkUnit` record, fetches its corresponding raw content file from GCS, and parses it, creating a new `ParsedObject` file (e.g., a JSON) in a "parsed objects" GCS bucket, organized by project (`/{projectId}/{documentId}/...`).
  3. It then updates the corresponding `WorkUnit` in Firestore with the GCS URI of the newly created `ParsedObject`.
  4. Finally, each `ParserService` creates a `CompletionTrackerTask` (with `projectId` and `documentId` context) in a dedicated sequential Cloud Tasks queue.

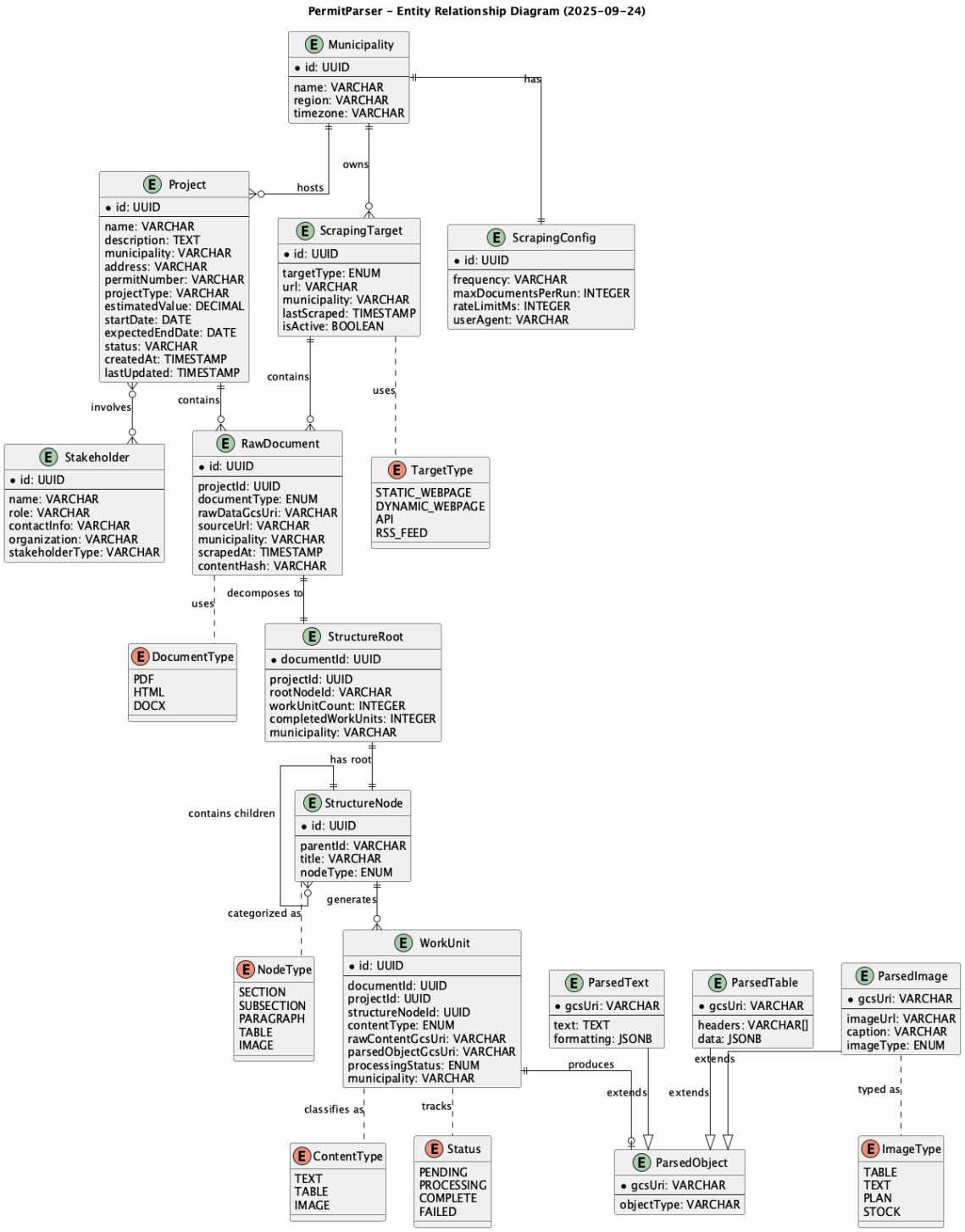
Stage 3: Orchestrated Sequential Aggregation (Transactional Fan-In)

- **Rationale:** To guarantee data integrity and prevent race conditions when updating the shared `StructureRoot` document, this final aggregation must be executed sequentially.
- **Flow:**
  1. A dedicated `completion-tracker-queue` in Cloud Tasks is configured with `max_concurrent_dispatches: 1`, creating a managed, sequential processor.
  2. The queue delivers `CompletionTrackerTask` items one-by-one to the **CompletionTrackerService** (Cloud Run).
  3. The `CompletionTrackerService` performs a safe, atomic read-modify-write on the `StructureRoot` document, incrementing the `completedWorkUnits` counter.
  4. When `completedWorkUnits` equals `workUnitCount`, the service publishes the final `DocumentCompleteEvent`, which includes the `projectId`.



4. Core Data Models and Design Patterns

4.1. Cloud-Native Data Models



Business Entities (Project-Centric):

- `Project` : A **Firestore document** representing a real-world construction project. Contains project metadata (name, permit number, address, municipality, estimated value, project type, dates, status). This is the central business entity that owns documents and stakeholders.
- `Stakeholder` : A **Firestore document** representing individuals or organizations involved in a project (contractors, architects, property owners, city officials). Associated with projects via many-

to-many relationships.

Processing Infrastructure:

- **`RawDocument`:** A **Firestore document** with metadata pointing to the original document stored in GCS. Includes `projectId` to link it to its parent construction project.
- **`StructureRoot`:** A lightweight **Firestore document** serving as the root metadata entry for a processed document. Contains `projectId` for project context, the total `workUnitCount`, the `completedWorkUnits` counter, and the ID of the root `StructureNode`.
- **`StructureNode`:** A **Firestore document** within a top-level `structureNodes` collection. Each document represents a single node in the hierarchy (e.g., a section) and uses a `parentId` field to form the tree (Adjacency List pattern).
- **`WorkUnit`:** A **Firestore document** representing a single, stateful unit of work. Contains `projectId` and `documentId` for context, pointers to raw and parsed content in GCS, and the current `processingStatus`.
- **`ParsedObject`:** A **GCS Object** (e.g., a JSON file) containing the structured data extracted from a `WorkUnit`. Storing this in GCS separates the large data payloads from the queryable metadata in Firestore.

4.2. Key Design Patterns

- **Saga Pattern:** The end-to-end workflow is a distributed transaction managed by Cloud Tasks (orchestration) and Pub/Sub (choreography).
- **Fan-Out/Fan-In Pattern:** Work is fanned out in parallel via Pub/Sub and safely fanned back in sequentially via a configured Cloud Tasks queue for aggregation.
- **Strategy Pattern:** Used for scrapers, **decomposition strategies**, and parsers to allow for extensible, pluggable logic for different document types and sources.
- **Adjacency List Pattern:** Used to model the hierarchical `StructureNode` tree within Firestore, avoiding single-document size limits and enabling scalable reads and writes.

5. Key Architectural Decisions & Justifications

- **Critical Race Condition Fix:** The primary risk of concurrent writes to the shared `StructureRoot` is mitigated at the infrastructure level by using a sequential Cloud Tasks queue for the aggregation step. This is a deliberate design choice to guarantee data consistency.
- **Firestore Adjacency List for Tree Structure:** The document's hierarchy is modeled as a collection of `StructureNode` documents linked by a `parentId`. This pattern is native to NoSQL and avoids Firestore's 1 MiB single-document size limit and write contention issues that would arise from nesting the entire tree in one document. It provides a scalable indexing solution.
- **Separation of Metadata (Firestore) and Data (GCS):** Firestore is used for what it excels at: storing and querying structured metadata and indices (`Project`, `WorkUnit`). Google Cloud Storage is used for what it excels at: storing large, immutable data blobs (original documents, raw content chunks, and `ParsedObject`s). This separation is cost-effective and highly scalable.
- **Serverless-First Compute:** Using Cloud Run and Cloud Functions minimizes operational overhead and cost by scaling to zero when idle and scaling out automatically under load, aligning with the event-driven nature of the pipeline.
- **Data Lifecycle and Retention:** The `WorkUnit` Firestore documents serve as critical "flight recorders" for debugging, auditing, and re-processing. To manage costs, a **retention policy will be implemented** using Firestore's Time-to-Live (TTL) feature to automatically delete these records after a defined period (e.g., 90 days). A similar policy will be applied to the intermediate "raw work unit" objects in GCS.